Problem statement - We analyze automotive diagnostic report files containing DTCs and environmental data from various vehicle models and software releases. We manually review DTCs from my ECU i.e., UCAP-10 and related ECUs (like BCP, MGU etc.) to determine root causes. The process is repetitive, time-consuming and requires correlation across multiple ECUs and historical datasets. We can leverage AI to automatically analyze large volumes of such reports, identify DTC trends across software releases, detect recurring issues after specific releases, correlate faults between interacting ECUs, and generate actionable diagnostic insights.

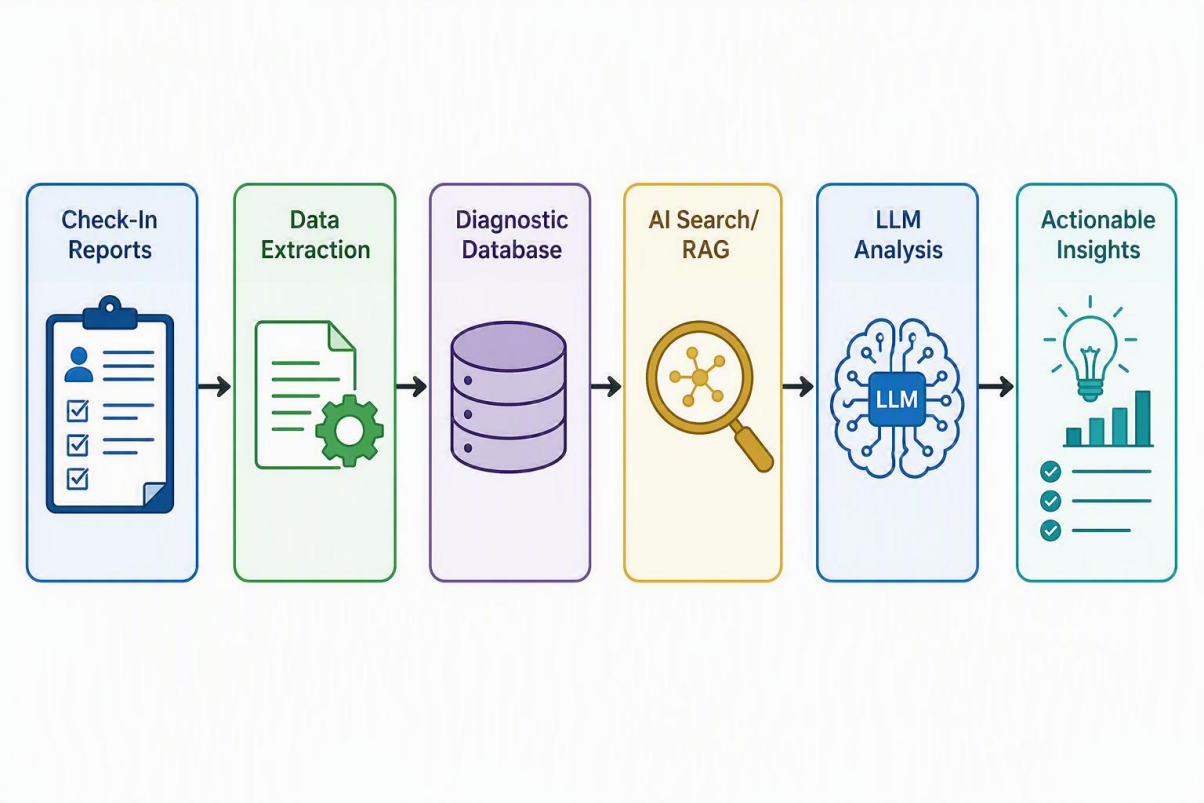
Business Benefits

- Reduce analysis time from 20–30 minutes to under a minute
- Automate DTC trend analysis across releases
- Generate AI-assisted root cause recommendations
- Correlate faults across interacting ECUs
- Detect software regressions early
- Enable predictive failure analysis
- Scale for future service packs & can be used across different departments

This is actually a very strong AI use case because we already have

- Large volumes of historical diagnostic reports
- Structured/semi-structured data (DTCs, timestamps, ECU names, environmental conditions)
- Expert knowledge that can be used to train the system
- A repetitive manual analysis process

Flow chart



We can approach it at **3 maturity levels**, rather than jumping directly to GenAI.

Level 1: Build a DTC Intelligence Database

Before AI, let's first create a centralized database from all check-in files.

Extract fields such as:

```
Vehicle Model
VIN (if allowed)

SW Release
SW Variant
Build Date

ECU Name
ECU Supplier

 DTC
 DTC Status
 Frequency Counter

 Environmental Data:
 - Battery Voltage
 - Engine Speed
 - Vehicle Speed
 - Temperature
 - Mileage
 - Ignition State
 - CAN timeout counters
 etc.

 Timestamp
```

And store it in:

- SQL Server
- Azure SQL
- Databricks
- PostgreSQL

Once all files are indexed, basic analytics become available.

Examples:

DTC trend by release

```sql
SELECT
    sw_release,
    dtc,
    COUNT(*)
FROM diagnostics
GROUP BY sw_release, dtc
```

We can immediately answer:

- Which DTC increased after release R21.4?
- Which DTC disappeared after R22.1?
- Which vehicle models have highest occurrence?

Level 2: AI-Assisted Root Cause Intelligence

This is where AI becomes valuable.

Create a knowledge base containing:

Historical analyses

Example:

```
DTC: U010000

Observed:
- Appears after SW 2.4.15
- Gateway ECU reports CAN timeout
- Battery voltage < 9V

Past RCA:
Wakeup sequence timing issue.

 Fix:
 SW 2.4.19
```

Engineering documents

- DTC definitions

- Failure mode descriptions
- ECU interface specifications
- CAN/LIN signal mapping
- Release notes
- Known issues database
- Jira defect database

Then we can use an LLM with RAG (Retrieval Augmented Generation).

Workflow:

```
Checkin File
        |
        v
DTC Extraction
        |
        v
Search Historical Cases
        |
        v
 LLM Analysis
         |
         v
 Suggested RCA
```

Output:

```
Detected DTC:
U010000

Observed in:
SW 2.4.15
SW 2.4.16
SW 2.4.17

Appeared after:
 SW 2.4.14

 Historical Similar Cases:
 DEFECT-1245
 DEFECT-1302

 Possible Causes:
```

```
85% CAN communication delay
10% Low voltage startup
5% ECU reset

Recommended checks:
1. Gateway logs
2. Voltage history
3. CAN bus utilization
```

Level 3: Cross-ECU Correlation Engine

This is probably your biggest pain point today.

Currently:

```
ECU A DTC found
        ↓
Open ECU B report
        ↓
Open ECU C report
        ↓
Manually correlate
```

AI can automate this.

Example:

Input

Powertrain ECU

```
U012100
Lost communication with BCM
```

BCM

```
B100100
Reset due to undervoltage
```

Gateway

```
U000100
CAN bus errors
```

Instead of showing three separate DTCs, AI generates:

```
Correlation Analysis:
```

```
2
3  Primary Event:
4  BCM reset due to undervoltage.
5
6  Secondary Effects:
7  - Powertrain ECU reported loss of BCM communication.
8  - Gateway reported CAN bus disturbances.
9
10  Probable Root Cause:
11  Low voltage event.
12
13  Confidence:
14  92%
```

This is something traditional dashboards cannot do well but LLMs are excellent at.

Level 4: Release Regression Detection

We can build release-wise intelligence.

The system continuously tracks:

```
1  DTC
2  Release
3  Occurrence Count
4  Vehicle Count
5  Severity
```

Example output:

```
1  DTC P123456
2
3  Release 4.1.1 : 3 cases
4  Release 4.1.2 : 5 cases
5  Release 4.1.3 : 48 cases
6  Release 4.1.4 : 62 cases
```

AI Insight:

```
1  Regression suspected.
2
3  Occurrence increased 12x after SW 4.1.2.
4
5  Affected vehicles:
```

```
SUV-A
SUV-B

Most common environmental conditions:
 Engine temperature < 10°C

 Potential related changes:
 Feature XYZ introduced in 4.1.3.
```

This is extremely valuable for release quality monitoring.

Level 5: Predictive Failure Analysis

Once we have thousands of historical reports.

Machine learning models can learn:

```
Pattern:
DTC A
+
DTC B
+
Voltage Dip

⇒ DTC C occurs within 2 weeks
```

Benefits:

- Early warning
- Warranty reduction
- Faster debugging

Example:

```
Current report risk assessment:

78% probability of:
P045612 appearing later.

Reason:
Pattern matches 124 historical vehicles.
```

Technology Stack

Data Layer

```
Azure Data Lake
Azure SQL
Databricks
```

AI Layer

```
Azure OpenAI
GPT-4o / GPT-5
RAG
```

Search Layer

```
Azure AI Search
```

Visualization

```
Power BI
```

A Practical Copilot Solution

We could build an internal "Diagnostic Copilot".

Engineer uploads:

```
vehicle_report.txt
```

Copilot returns:

```
Summary:
5 DTCs found.

Critical:
U010000

Trend:
Observed in 42 vehicles after SW 3.2.11.

 Related ECUs:
 Gateway
 BCM

 Historical Defects:
```

```
BUG-2345
BUG-3188

Most Probable RCA:
CAN wakeup timing issue.

Confidence:
87%
```

The analysis that currently takes 20-30 minutes could potentially be reduced to under a minute.

What we can build first

For fastest ROI:

Phase 1 (4-6 weeks)

- Parse all check-in files
- Create DTC database
- Power BI dashboard
- Release-wise DTC trending

Phase 2

- Add historical Jira defects
- Add release notes
- Build RAG-based Diagnostic Copilot

Phase 3

- Multi-ECU correlation engine
- Automated RCA suggestions
- Regression detection

This phased approach usually delivers value much faster than trying to build a full AI solution from day one and aligns very well with automotive diagnostics workflows.

Initial steps

Utilizing CG assets, create a POC based on the dummy checkin files & demonstrate to CG leadership how this AI solution can provide the diagnostics insights. We can deploy below CG resources reporting to me who are currently on bench –

- Vasanth K
- Guru Mohan M

Once we have a confidence that it is working as expected, we can approach BMW to showcase this AI use case.